EcoOps API Contract

Project: EcoOps --- Agentic Campus Sustainability Platform
Hackathon: Kaushal 8.0, PS-04 Sustainable Tech Without Limits
Version: 1.0
Frontend: React + Vite
Backend: Node.js + Express
Database: MongoDB with Mongoose

> This file is the shared contract for the frontend and backend developers. Both sides must follow the same paths, field names, status codes, and response shapes. If a change is necessary, update this file and tell both developers before implementing it.

1. Scope and implementation priorities

MVP --- implement first

1. Demo users and role switching (simple hackathon demo mode; full password authentication is not required by the PRD).
2. Complaint submission with optional photo and automatic agent trigger.
3. Usage dashboard data and anomaly list.
4. Ticket board and ticket status/resolution updates.
5. Manual agent run.
6. Agent activity log.
7. Approval queue and approve/reject decisions.
8. In-app notifications.
9. Impact counters and savings after a ticket is resolved.
10. Simulated usage-data generation and deterministic anomaly detection.

Should-have --- only after the MVP works

- Agent chat: `POST /api/agent/chat`
- Weekly sustainability report: `POST /api/reports/weekly`
- Optional image analysis of complaint photos.

Explicit project constraints

- Usage data is simulated and must be labelled as simulated in the UI.
- Notifications are stored in the app; do not integrate real email, SMS, or WhatsApp.
- Agent tools may only access this application's own database.
- High-impact actions require human approval.

- Agent tool arguments must be validated on the server.
- Limit the number of tool calls/steps per agent run and log every tool call.
- The demo may use seeded Student, Facility Staff, and Admin users with a simple role switcher rather than full production authentication.

2. Base URLs and conventions

- Local frontend: `http://localhost:5173`
- Local backend: `http://localhost:5000`
- Local API base: `http://localhost:5000/api`
- Frontend environment variable: `VITE_API_BASE_URL=http://localhost:5000/api`
- All JSON request and response bodies use UTF-8 JSON, except complaint photo upload, which uses `multipart/form-data`.
- All timestamps are ISO 8601 UTC strings.
- MongoDB ObjectIds are returned as strings named `id`; clients must not depend on MongoDB's internal `_id` field.
- List endpoints return `{ "success": true, "data": { "items": [], "pagination": {} } }` unless an endpoint below specifies a different shape.
- Do not return passwords, API keys, database connection strings, or other secrets.

3. Standard response and error format

Success

```
{
  "success": true,
  "message": "Request completed",
  "data": {}
}
```

Error

```
{
  "success": false,
  "message": "A human-readable error message",
  "errors": []
}
```

`errors` may contain validation details:

```
[
  { "field": "description", "message": "Description is required" }
]
```

Use these status codes consistently: - `200` --- successful read/update/action - `201` --- resource created - `400` --- invalid input - `401` --- no valid demo user context or authentication, if applicable - `403` --- current role is not permitted to perform the action - `404` --- resource does not exist - `409` --- conflicting operation - `413` --- uploaded file too large - `415` --- unsupported file type - `500` --- unexpected server error - `503` --- required service unavailable

4. Demo user and role context

The PRD recommends seeded demo users and a role switcher instead of full authentication. Implement this simple mode consistently.

`GET /api/demo/users`

Returns the available seeded demo users. Do not return credentials or secrets.

Response `200`:

```
{
  "success": true,
  "data": {
    "users": [
      { "id": "demo-student", "name": "Aarav Student", "role": "student" },
      { "id": "demo-staff", "name": "Facilities Staff", "role": "staff" },
      { "id": "demo-admin", "name": "Sustainability Admin", "role": "admin" }
    ]
  }
}
```

Frontend sends `X-Demo-User-Id: demo-student`, `demo-staff`, or `demo-admin` on API requests. Backend resolves the ID against seeded users and enforces role permissions. Reject unknown IDs. This is a demo-only identity mechanism, not production authentication.

Role values: `student | staff | admin`.

Permissions: - `student`: create complaints; view own complaints and general dashboard/impact data. - `staff`: view assigned tickets; update permitted ticket statuses and add resolution notes; view relevant notifications. - `admin`: view all dashboard data, tickets, logs, approvals and reports; run the agent; approve/reject actions. - The agent is a backend service, not a client-controlled role.

5. Health

`GET /api/health`

No authentication required.

Response `200`:

```
{
  "success": true,
  "data": {
    "api": "ok",
    "database": "connected",
    "timestamp": "2026-10-12T08:00:00.000Z"
  }
}
```

If the API is running but MongoDB is unavailable, return `503` and `database: "disconnected"`. Do not report the entire service as healthy when database-backed features cannot work.

6. Complaints

`POST /api/complaints`

Creates a complaint and triggers an agent run.

Headers: - `X-Demo-User-Id: demo-student` (or another valid demo user) - For photo upload, use `multipart/form-data`; otherwise JSON is acceptable.

Fields: - `type` --- required; enum: `water_leak | lights_ac_left_on | waste | other` - `buildingId` --- required building ID from the API - `location` --- required room/floor/location text, e.g. `2nd floor` - `description` --- required, 10--2000 characters - `photo` --- optional image file; accepted types JPEG, PNG, WebP; maximum 5 MB

Response `201`:

```
{
  "success": true,
  "message": "Complaint submitted",
  "data": {
    "complaint": {
      "id": "complaint-id",
      "userId": "demo-student",
      "type": "water_leak",
      "buildingId": "building-hostel-b",
      "location": "2nd floor",
      "description": "Tap leaking near the washroom",
      "imageUrl": null,
      "status": "submitted",
      "createdAt": "2026-10-12T08:00:00.000Z"
    },
    "agentRun": {
      "id": "agent-run-id",
      "status": "queued"
    }
  }
}
```

Agent processing may be synchronous or queued, but the response contract must remain stable. If queued, the activity-log endpoint should show progress.

`GET /api/complaints`

Query parameters: - `status` --- optional complaint status - `buildingId` --- optional - `page` --- optional, default `1` - `limit` --- optional, default `20`, maximum `100`

Students/faculty see their own complaints; staff/admin permissions follow the PRD and backend policy.

Response `200`:

```
{
  "success": true,
  "data": {
    "items": [],
    "pagination": { "page": 1, "limit": 20, "total": 0, "pages": 0 }
  }
}
```

`GET /api/complaints/:id`

Returns a single complaint if the current demo user is allowed to view it.

7. Buildings and simulated usage

`GET /api/buildings`

Returns buildings used by complaint forms and dashboards.

Response `200`:

```
{
  "success": true,
  "data": {
    "buildings": [
      { "id": "building-hostel-b", "name": "Hostel B", "type": "hostel" }
    ]
  }
}
```

`GET /api/usage`

Query parameters: - `buildingId` --- optional - `resource` --- optional enum: `electricity | water` - `from` and `to` --- optional ISO date/time range - `interval` --- optional; `hour` or `day`, default `hour`

Response `200`:

```
{
  "success": true,
  "data": {
    "simulated": true,
    "unitByResource": { "electricity": "kWh", "water": "litres" },
    "series": [
      {
        "buildingId": "building-hostel-b",
        "buildingName": "Hostel B",
        "resource": "water",
        "timestamp": "2026-10-12T08:00:00.000Z",
        "value": 120.5,
        "baseline": 80.0,
        "isAnomaly": true
      }
    ]
  }
}
```

The backend owns data generation and anomaly detection. Detection must be deterministic and explainable (e.g. rolling baseline per building and hour plus z-score or a baseline multiple). Do not ask the AI model to calculate the numeric anomaly itself.

`POST /api/usage/generate`

Admin/demo operator only. Generates or refreshes the demo dataset: about five buildings, hourly readings over fourteen days, and five to six injected anomalies. Must avoid accidental duplicate data on repeated calls (clear-and-regenerate a known demo dataset or use deterministic upserts).

Response `200`:

```
{
  "success": true,
  "message": "Demo usage data generated",
  "data": { "buildingsCreated": 5, "readingsCreated": 1680, "anomaliesDetected": 6, "simulated": tru
}
```

Adjust the readings count to the actual resources generated; do not hardcode an inaccurate count.

`GET /api/anomalies`

Query parameters: - `buildingId` --- optional - `resource` --- optional `electricity | water` - `status` --- optional `open | linked | resolved` - `from`, `to` --- optional ISO timestamps

Response `200`:

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "anomaly-id",
        "buildingId": "building-hostel-b",
        "buildingName": "Hostel B",
        "resource": "water",
        "timestamp": "2026-10-12T02:00:00.000Z",
        "observedValue": 220,
        "baselineValue": 80,
        "score": 3.1,
        "reason": "Water usage is 2.75x the expected baseline",
        "status": "open"
      }
    ]
  }
}
```

8. Agent

`POST /api/agent/run`

Admin only. Manually triggers the agent to inspect open complaints and/or unhandled anomalies.

Request:

```
{
  "trigger": "manual",
  "buildingId": null,
  "anomalyId": null,
  "complaintId": null
}
```

`trigger` enum: `manual | complaint | anomaly`. For manual runs, other IDs may be omitted. A complaint submission automatically triggers the agent; a newly detected anomaly must also trigger it or enter a queue.

Response `202`:

```
{
  "success": true,
  "message": "Agent run started",
  "data": {
    "runId": "agent-run-id",
    "status": "queued"
  }
}
```

`GET /api/agent/runs/:id`

Returns run status: `queued | running | completed | failed`, start/end timestamps, and a concise summary.

`GET /api/agent/logs`

Query parameters: - `limit` --- default `50`, maximum `100` - `cursor` --- optional pagination cursor - `runId` --- optional filter

Response `200`:

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "log-id",
        "runId": "agent-run-id",
        "timestamp": "2026-10-12T08:00:00.000Z",
        "trigger": "complaint",
        "step": 1,
        "thought": "A water complaint matches a recent water-usage anomaly in the same building.",
        "tool": "detect_anomalies",
        "result": "Found one relevant anomaly",
        "status": "success"
      }
    ],
    "nextCursor": null
  }
}
```


`thought` must be a short user-facing rationale, not private chain-of-thought. Log the observation, action decision, tool name, sanitized result, status, and timestamp. Never log API keys or secrets.

Agent tool constraints

The model may select only registered backend tools. The backend must validate tool arguments, enforce authorization and approval rules, prevent duplicate open tickets for the same issue, and cap each run (suggested maximum: 8 tool calls). High-impact actions must create an approval rather than execute immediately.

Required internal tools: - `get_usage(building, resource, range)` - `detect_anomalies(building, range)` - `get_open_tickets(location)` - `create_ticket(location, issue, priority, cause, action)` - `assign_staff(ticketId, team)` - `send_notification(to, message)` - `request_approval(action, reason)` - `calculate_impact(ticketId)`

Optional tools after MVP: - `analyze_image(imageId)` - `generate_report(range)`

9. Tickets

Ticket status enum: `open | assigned | in_progress | resolved`. Priority enum: `low | medium | high | critical`.

`GET /api/tickets`

Query parameters: - `status` --- optional - `priority` --- optional - `buildingId` --- optional - `assignee` --- optional - `page` and `limit`

--- optional

Staff see tickets assigned to them/their team; admin can view all tickets. The UI may display Kanban columns by grouping these statuses.

Response `200`:

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "ticket-id",
        "complaintId": "complaint-id",
        "buildingId": "building-hostel-b",
        "location": "2nd floor",
        "issue": "Water leak",
        "priority": "high",
        "cause": "Possible continuous leak indicated by overnight water anomaly",
        "recommendedAction": "Inspect and repair the tap",
        "assignee": { "team": "plumbing", "userId": "demo-staff", "name": "Facilities Staff" },
        "status": "assigned",
        "resolutionNote": null,
        "impact": null,
        "createdAt": "2026-10-12T08:00:00.000Z",
        "updatedAt": "2026-10-12T08:00:00.000Z"
      }
    ],
    "pagination": { "page": 1, "limit": 20, "total": 1, "pages": 1 }
  }
}
```

`GET /api/tickets/:id`

Returns one ticket if permitted.

`POST /api/tickets`

Optional manual ticket creation. Admin/staff only. Request:

```
{
  "buildingId": "building-hostel-b",
  "location": "2nd floor",
  "issue": "Water leak",
  "priority": "medium",
  "cause": "Reported by facilities staff",
  "recommendedAction": "Inspect tap",
  "assigneeTeam": "plumbing"
}
```

Response `201` with the created ticket.

`PATCH /api/tickets/:id`

Staff/admin only. Only accept the supported fields:

```
{
  "status": "in_progress",
  "resolutionNote": "Technician is inspecting the tap"
}
```

When status changes to `resolved`, require a non-empty `resolutionNote`, calculate impact through deterministic backend formulas, store the impact result, and update the global impact summary. Do not allow a client to submit arbitrary savings numbers. A high-impact repair/action must follow the approval workflow before execution.

Response `200` returns the updated ticket.

10. Approvals

`GET /api/approvals`

Admin only. Optional query parameter `status=pending|approved|rejected`.

Response `200`:

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "approval-id",
        "ticketId": "ticket-id",
        "action": "Authorize high-priority plumbing repair",
        "reason": "The issue may cause significant water waste",
        "status": "pending",
        "requestedAt": "2026-10-12T08:00:00.000Z",
        "decidedAt": null,
        "decidedBy": null,
        "decisionNote": null
      }
    ]
  }
}
```

`POST /api/approvals/:id/decision`

Admin only. Request:

```
{
  "decision": "approved",
  "decisionNote": "Approved for facilities team inspection"
}
```

`decision` enum: `approved | rejected`. Only pending approvals can be decided. Repeating a decision returns `409`. Store who decided and when. Approval must authorize only the documented action; it must not permit arbitrary tool execution.

Response `200` returns the updated approval and any resulting action status.

11. Notifications inbox

`GET /api/notifications`

Returns notifications visible to the current demo user. Query parameters: `unreadOnly=true|false`, `limit` (default 50, max 100).

Response `200`:

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "notification-id",
        "to": "demo-staff",
        "message": "A new plumbing ticket has been assigned to you.",
        "type": "ticket_assigned",
        "read": false,
        "createdAt": "2026-10-12T08:00:00.000Z"
      }
    ],
    "unreadCount": 1
  }
}
```

`PATCH /api/notifications/:id`

Marks a notification as read. Request:

```
{ "read": true }
```

A user may only update notifications addressed to them; admin access may be granted for demo oversight.

12. Impact metrics

`GET /api/impact`

Returns aggregate campus impact calculated from resolved tickets and their recorded impact values.

Response `200`:

```
{
  "success": true,
  "data": {
    "issuesDetected": 6,
    "issuesResolved": 2,
    "electricitySavedKwh": 18.5,
    "waterSavedLitres": 420,
    "moneySavedInr": 350,
    "co2AvoidedKg": 12.4,
    "assumptions": {
      "electricityTariffInrPerKwh": 8,
      "waterCostInrPerLitre": 0.05,
      "electricityEmissionKgCo2PerKwh": 0.67
    },
    "simulated": true
  }
}
```

These numbers are illustrative only. Backend must calculate from stored readings, resolution time, and documented constants. The UI must disclose that data and savings are simulated and show the actual assumptions used. Do not claim measured real-world savings.

13. Optional agent chat

`POST /api/agent/chat`

Admin/staff demo access. Request:

```
{
  "message": "What are the top 3 sustainability problems today?"
}
```

Response `200`:

```
{
  "success": true,
  "data": {
    "answer": "The three highest-priority issues are...",
    "sourceIds": ["anomaly-id", "ticket-id"],
    "simulated": true
  }
}
```

The agent must answer using application data and tool results, not invented facts. If required data is unavailable, say so. Never expose secrets or internal prompts.

14. Optional weekly report

`POST /api/reports/weekly`

Admin only. Request:

```
{
  "weekEnding": "2026-10-11"
}
```

Response `200`:

```
{
  "success": true,
  "data": {
    "reportId": "report-id",
    "title": "Weekly Campus Sustainability Report",
    "period": { "from": "2026-10-05", "to": "2026-10-11" },
    "summary": "Simulated usage and resolved issue summary...",
    "metrics": {
      "issuesDetected": 6,
      "issuesResolved": 2,
      "electricitySavedKwh": 18.5,
      "waterSavedLitres": 420,
      "moneySavedInr": 350,
      "co2AvoidedKg": 12.4
    },
    "simulated": true,
    "generatedAt": "2026-10-12T08:00:00.000Z"
  }
}
```

15. Optional image analysis

If implemented, complaint photos are uploaded through `POST /api/complaints`. The backend stores the image using a safe

storage method and returns an `imageUrl` or `imageId`. Never trust a client-supplied URL as a stored file. Restrict file size and content type. If vision is unavailable, complaint submission must still work and the UI should show that image analysis was skipped.

16. Database entity minimum fields

- `User`: name, role (`student | staff | admin`), optional team; seeded demo records.
- `Building`: name, type.
- `Reading`: buildingId, resource (`electricity | water`), timestamp, value, simulated.
- `Anomaly`: buildingId, resource, timestamp, observedValue, baselineValue, score, reason, status.
- `Complaint`: userId, type, buildingId, location, description, imageUrl/imageId, status, createdAt.
- `Ticket`: complaintId, buildingId, location, issue, priority, cause, recommendedAction, assignee, status, resolutionNote, impact, timestamps.
- `AgentRun`: trigger, status, start/end timestamps, summary.
- `AgentLog`: runId, timestamp, trigger, step, thought/rationale, tool, result, status.
- `Approval`: ticketId/action, reason, status, requestedAt, decidedAt, decidedBy, decisionNote.
- `Notification`: to, message, type, read, createdAt.
- `Impact`: derived from resolved tickets; may be computed on read or maintained safely in a consistent aggregate.

Use indexes for common filters such as timestamp, buildingId/resource, status, assignee, and unread notifications as appropriate.

17. Frontend integration requirements

- Frontend base URL comes only from `import.meta.env.VITE_API_BASE_URL`.
- Add the selected `X-Demo-User-Id` header to requests; update it when the role switcher changes.
- Centralize requests and parse the standard response format.
- Do not assume success solely because HTTP returned a response; check `success`.
- Display loading, empty, and error states.
- Refresh dashboard data after complaint creation, ticket resolution, approval decisions, and agent runs.

- Poll `/api/agent/logs` or use Server-Sent Events if implemented. Polling is acceptable for the hackathon.
- Do not store secrets in frontend environment variables.
- Mark simulated readings and calculated impact visibly in the UI.

18. Acceptance tests for integration

1. `GET /api/health` reports API and database status correctly.
2. Demo role switcher loads seeded users and changes the role header.
3. Student submits a complaint with and without a photo.
4. Complaint triggers an agent run and creates activity-log entries.
5. Usage dashboard loads time-series data and visibly marks it simulated.
6. Anomalies are generated deterministically and returned with explanations.
7. Agent creates a ticket, assigns a team, and sends an in-app notification.
8. High-impact action creates a pending approval; admin can approve/reject it.
9. Staff updates a ticket and adds a resolution note.
10. Resolving a ticket updates impact metrics using backend-calculated values.
11. Role restrictions are enforced on the backend, not only hidden in the UI.
12. Invalid IDs, invalid fields, unsupported photos, and database failures return the documented error shape.
13. The core demo still works if optional vision, chat, or weekly-report features are not implemented.

19. Change-control rule

Neither frontend nor backend developer may silently alter this contract. If a mismatch is discovered: 1. Record the mismatch. 2. Agree on the intended behavior. 3. Update this file. 4. Change the relevant code. 5. Retest the affected integration.

MVP priority: complaint → agent run → activity log → ticket/assignment/notification → approval if high-impact → staff resolves ticket → impact counter updates.